

ServiLoc

Organisation frontend — 16 écrans prioritaires · 6 membres · 4 semaines

Équipe	6 membres (1 référent, 1 intermédiaire, 4 débutants)
Durée	4 semaines · 2–3h/jour effectif par membre
Scope	16 écrans prioritaires (P1 + P2 + 2 P3)
Stack	React + Vite + Tailwind CSS + React Router v6 + Axios
Document	31 mai 2026 — à conserver dans le repo à la racine

M1 = Krisan, M2 = Tresor, M3 = Archange, M4 = Kenfack, M5 = Patricia, M6 = Cynthia

RÉPARTITION DES 6 MEMBRES

Référent technique (M1)	Niveau avancé
Setup projet complet, composants core (/src/components/), dashboard admin, validation prestataire admin, review de toutes les PR, déblocage équipe.	
Setup projet • Composants core (UNIQUEMENT) • Admin dashboard • Admin validation • PR review	
Membre 2	Niveau intermédiaire
Écrans d'inscription, vérification OTP, dashboard client, notation prestataire.	
Inscription • OTP (timer + renvoi) • Dashboard client • Notation prestataire	
Membre 3	Niveau débutant
Formulaire nouvelle demande, liste des demandes disponibles (prestataire), formulaire de création de devis.	
Nouvelle demande • Demandes disponibles • Créer devis	
Membre 4	Niveau débutant
Dashboard prestataire, démarrage mission + étapes de progression, formulaire signalement litige.	
Dashboard prestataire • Démarrer mission • Signaler litige	
Membre 5	Niveau débutant
Page connexion (client / prestataire), page connexion admin, écran suivi de mission temps réel, états chargement globaux.	
Connexion client/presta. • Connexion admin • Suivi mission • Loaders & empty states	
Membre 6	Niveau débutant + rôle QA
Gestion utilisateurs admin (liste + suspension), litiges admin, showcase composants S1, tests QA hebdomadaires.	
Gestion users admin • Litiges admin • Showcase S1 • QA hebdomadaire	

16 ÉCRANS PRIORITAIRES

Prio	Profil	Écran	Membre	UC
P1	Auth	Inscription / Connexion	M2 + M5	UC1
P1	Auth	Vérification OTP	M2	UC1
P1	Auth	Connexion Admin	M5	UC29
P1	Client	Dashboard Client	M6	UC2
P1	Client	Nouvelle Demande	M3	UC2-3-4
P1	Prestataire	Dashboard Prestataire	M4	UC28
P1	Prestataire	Demandes Disponibles	M3	UC19-21
P1	Admin	Dashboard Admin	M1	UC29
P2	Client	Suivi de Mission	M5	UC9-10
P2	Client	Notation Prestataire	M2	UC11
P2	Prestataire	Créer Devis	M3	UC22-23
P2	Prestataire	Démarrer Mission	M4	UC24
P2	Admin	Validation Prestataire	M1	UC30
P2	Admin	Gestion Utilisateurs	M6	UC31
P3	Client	Signaler Litige	M4	UC12
P3	Admin	Litiges Admin	M6	UC34

PLAN D'EXÉCUTION — 4 SEMAINES

Semaine 1 Fondations — personne ne code de pages

Objectif unique : avoir tous les composants prêts pour que les débutants puissent coder sans se bloquer. Si quelqu'un commence une page avant la fin de S1, il devra tout réécrire avec les vrais composants.

Référent (M1)	J1–J2 : Vite + Tailwind + React Router, structure dossiers, .env.development J3–J5 : Button (variants), Card, Input, Modal, Spinner, Badge, Avatar — dans /src/components/
Membre 2	J1–J2 : Git flow — branches, conventions commits, PR workflow J3–J5 : Sidebar (3 variantes client/presta/admin) + Topbar + design tokens CSS
Membre 3	J1–J3 : Apprendre Tailwind (classes utilitaires, responsive) J4–J5 : Composant FormField réutilisable + validation de base
Membre 4	J1–J3 : Setup Axios + mockSwitch.js + apiClient.js avec interceptors JWT J4–J5 : StatusBadge, RatingStars, PriceDisplay (XAF)
Membre 5	J1–J3 : Apprendre React Router v6 (routes imbriquées, params) J4–J5 : Routes app + AuthGuard + PrivateRoute (protection par rôle)

Membre 6	J1–J3 : Importer et explorer les données mock (src/data/ J4–J5 : Page /showcase — affiche tous les composants de l'équipe
-----------------	--

Livrables :

- ✓ localhost/showcase affiche tout sans erreur
- ✓ 0 page métier — c'est normal et voulu
- ✓ Chaque membre a fait ≥ 3 commits sur sa branche
- ✓ API_CONTRACT.md versionné dans le repo

Semaine 2 Pages P1 avec données mock

8 pages P1 livrées avec données mock. Chaque page utilise **UNIQUEMENT** les composants de M1 — pas de duplication. Point de sync avec le backend vendredi.

Référent (M1)	Toute la semaine : Dashboard Admin + composant DataTable + StatCard En parallèle : review PR de chaque membre avant merge dans develop
Membre 2	J1–J3 : Écran inscription — form complet avec validation (email, tel, mot de passe) J4–J5 : Vérification OTP — champ 6 chiffres, timer 45s, bouton renvoi
Membre 3	J1–J3 : Nouvelle demande — formulaire multi-étapes (catégorie, description, zone, budget) J4–J5 : Demandes disponibles — liste avec filtres zone/catégorie, carte par demande
Membre 4	J1–J3 : Dashboard prestataire — stats + missions actives + gains du mois J4–J5 : Affiner avec données mock complètes + états vides
Membre 5	J1–J3 : Page connexion client/prestataire — email + mot de passe + gestion erreurs J4–J5 : Page connexion admin/SC — même base, branding différent
Membre 6	J1–J3 : Dashboard client — résumé demandes + dépenses + missions J4–J5 : Tester navigation complète + rapport bugs visuels à l'équipe

Livrables :

- ✓ 8 pages P1 rendues sans erreur console
- ✓ Navigation fonctionnelle entre toutes les pages
- ✓ Point sync backend vendredi 30 min
- ✓ Données mock partout — aucun appel API réel

Semaine 3 Pages P2 + connexion API progressive

16 écrans complets. M1 intègre l'auth JWT réelle. Les autres connectent leurs endpoints au fur et à mesure que le backend livre. Fallback mock maintenu partout.

Référent (M1)	J1–J2 : Intégration auth JWT réelle — login → stockage token → AuthGuard actif J3–J5 : Validation prestataire admin (dossier + documents + accept/reject) + review PR
Membre 2	J1–J3 : Suivi de mission — barre progression, étapes, statut temps réel (SSE) J4–J5 : Notation prestataire — formulaire étoiles 1-5 + commentaire
Membre 3	J1–J3 : Créer devis — tableau matériaux dynamique, totaux auto, durée J4–J5 : Connecter GET /provider/demands/available à l'API réelle

Membre 4	J1–J3 : Démarrer mission + cases à cocher étapes + bouton terminer J4–J5 : Signaler litige — sélection motif, description, upload photos
Membre 5	J1–J3 : Connecter auth réelle (endpoints livrés S2 backend) J4–J5 : Ajouter loaders, messages d'erreur, empty states sur TOUTES les pages
Membre 6	J1–J3 : Gestion utilisateurs admin — tableau paginé, badge statut, suspend/réactiver J4–J5 : Litiges admin — liste avec métriques, détail litige, résolution

Livrables :

- ✓ 16 écrans tous fonctionnels
- ✓ Auth connectée à l'API réelle
- ✓ Flow complet client testé bout en bout
- ✓ Loaders et empty states présents partout

Semaine 4 Stabilisation + déploiement + démo

Aucune nouvelle feature. Uniquement corrections, intégration API restante, déploiement Vercel et préparation du scénario de démo. Ne jamais déployer le vendredi.

Référent (M1)	J1–J2 : Déploiement Vercel + configuration CORS production + variables d'env prod J3–J5 : Scénario démo documenté + README complet
Membre 2	J1–J3 : Connecter endpoints S3 backend (demandes, missions, notation) J4–J5 : Corrections bugs P1 uniquement
Membre 3	J1–J3 : Connecter devis + demandes disponibles API réelle J4–J5 : Responsive mobile — vérifier les 16 écrans sur 375px
Membre 4	J1–J3 : Connecter mission + litige API réelle J4–J5 : Polissage UX — hover states, transitions, focus rings
Membre 5	J1–J3 : Tests flows complets (inscription → demande → mission → notation) J4–J5 : Répétition scénario démo × 2 avec toute l'équipe
Membre 6	J1–J3 : Connecter admin API (users, litiges, dashboard) J4–J5 : Buffer imprévus + checklist QA finale (16 écrans × checklist)

Livrables :

- ✓ Application déployée sur Vercel
- ✓ Scénario démo répété 2 fois
- ✓ Tous les endpoints livrés connectés
- ✓ README à jour — installation en < 5 min

RÈGLES NON NÉGOCIABLES

Propriété des composants	Le référent (M1) est le SEUL à écrire dans /src/components/. Les autres membres utilisent les composants — ils ne les dupliquent pas, ne les réécrivent pas. Si un composant manque → demander à M1.
---------------------------------	--

Pas de pages avant S2	En semaine 1, personne ne code de page. Si quelqu'un commence sa page avant que les composants soient validés, il devra tout réécrire. La patience ici évite 2 jours de perte en S2.
Git — 1 branche par membre	Convention : feature/auth-m2, feature/dashboard-client-m6... Commits atomiques : 'feat(auth): add OTP countdown timer'. PR obligatoire pour merger dans develop. M1 valide avant merge.
Standup 15 min max chaque matin	Format unique : Fini / En cours / Bloqué. Si bloqué depuis plus de 2h → demander de l'aide immédiatement. Ne pas attendre le lendemain. Ne pas rester bloqué en silence.
VITE_USE MOCK=true jusqu'en S3	Tous les membres travaillent avec les mocks en S1 et S2. Seul M1 bascule les endpoints au fur et à mesure en S3. Chaque appel API garde son fallback mock dans le catch.
M6 est le QA officiel	En plus de ses pages, M6 teste les pages des autres chaque vendredi et reporte les bugs visuels dans un fichier BUGS.md dans le repo. Bugs P1 (bloque la démo) = correction obligatoire avant le lundi.
Jamais déployer le vendredi	Tous les déploiements Vercel se font lundi-jeudi en semaine 4. Le vendredi est réservé aux tests post-déploiement et corrections.
Un composant = un fichier = un seul responsable	Deux membres ne modifient jamais le même fichier en même temps. Si c'est nécessaire → se coordonner oralement avant, puis PR immédiat.

INTÉGRATION PROGRESSIVE AVEC LE BACKEND

Le principe central : on ne change jamais les imports dans les pages — seule la variable VITE_USE MOCK change, et les fonctions de service basculent endpoint par endpoint à partir de S3.

Semaine	VITE_USE MOCK	Qui bascule ?	Ce qui change
S1	true (tous)	—	Aucun appel API. Développement composants uniquement.
S2	true (tous)	—	Aucun appel API. Pages avec données mock.
S3	false (M1 seul)	M1 en premier puis les autres	Auth réelle en premier, puis endpoints livrés un par un. Fallback mock maintenu dans chaque catch.
S4	false (tous)	Chaque membre pour ses pages	Connecter tous les endpoints livrés. Fallback urgence en cas de panne Railway en démo.

Pattern à appliquer dans chaque service :

```
// .env.development VITE_USE MOCK=true ← S1/S2 VITE_USE MOCK=false ← S3/S4 //
src/services/clientService.js export async function getDashboard() { return
getMock(mockClientDashboard, // fallback toujours là () => axios.get('/client/dashboard')
// appel réel quand USE MOCK=false ) } // Dans la page – pattern try/catch obligatoire
getDashboard() .then(setData) .catch(() => setData(mockClientDashboard.data)) // fallback
urgence démo
```

STRUCTURE DU PROJET

```
src/ ■■■ components/ ← RÉFÉRENT TECHNIQUE UNIQUEMENT ■ ■■■ Button.jsx (variants:
primary, secondary, ghost, danger) ■ ■■■ Card.jsx ■ ■■■ Input.jsx ■ ■■■ Modal.jsx ■
■■■ Sidebar.jsx (props: role → client | provider | admin) ■ ■■■ Topbar.jsx ■ ■■■
Badge.jsx ■ ■■■ Avatar.jsx ■ ■■■ Spinner.jsx ■ ■■■ DataTable.jsx ■ ■■■ StatCard.jsx ■
■■■ StatusBadge.jsx (M4) ■ ■■■ RatingStars.jsx (M4) ■ ■■■ FormField.jsx (M3) ■■■
pages/ ■ ■■■ auth/ (M2 + M5) ■ ■■■ client/ (M2 + M6) ■ ■■■ provider/ (M3 + M4) ■ ■■■
admin/ (M1 + M6) ■■■ services/ ■ ■■■ mockSwitch.js (M4 – S1) ■ ■■■ apiClient.js (M4 –
S1) ■ ■■■ authService.js ■ ■■■ clientService.js ■ ■■■ providerService.js ■ ■■■
adminService.js ■ ■■■ sharedService.js ■■■ data/ ← Données mock (déjà générées) ■ ■■■
auth/ ■ ■■■ client/ ■ ■■■ provider/ ■ ■■■ admin/ ■ ■■■ shared/ ■■■ styles/ ■■■
tokens.css (M2 – design tokens ServiLoc)
```

Document ServiLoc — Équipe Frontend · Préparé avec Claude · Mai 2026 · Version 1.0 — à conserver à la racine du repo GitHub